

OpenFOAM MHD-VoF 求解器集合

算法与代码结构分析

基于 OpenFOAM v1912 求解器源码分析

源码审计版：2026 年 6 月

摘要

本文档基于仓库源码而非仅依据求解器名称，对 OpenFOAM MHD-VoF 求解器集合进行算法、数学公式、调用关系和使用方法审计。仓库面向电渣重熔（ESR）中的两相流、电磁力、焦耳热和共轭传热问题，组合了 MULES/VoF-Library、isoAdvector、PIMPLE、EOF（OpenFOAM-Elmer）以及可变密度湍流模型。文档以完整 MHD-CHT-Iso-Cpad 修改版和随仓库提供的 ESR 算例为主线，同时说明 README 中列出的 17 个变体之间的差异。所有公式均区分“连续物理模型”“源码离散残差”和“需要进一步验证的实现假设”；仓库作者也明确声明这些求解器尚未完成数学验证或工程验证。

目录

1 引言	3
1.1 项目背景	3
1.2 依赖项目	3
1.3 审计范围与阅读约定	3
1.4 命名约定	4
2 求解器架构总览	4
2.1 求解器分类	4
2.2 目录结构	5
3 控制方程	5
3.1 相分数方程（VOF 方法）	6
3.2 MULES 人工压缩与几何输运的区别	6
3.3 质量守恒方程	6
3.4 动量方程（含 MHD 洛伦兹力）	7
3.5 能量方程	7
3.5.1 流体区温度方程	7
3.5.2 固体区能量方程	8
3.6 压力方程（PIMPLE 算法）	8
3.7 湍流模型	9
3.7.1 CHT 路径的可压缩两相湍流	9
3.7.2 单区域 varRho 路径	9

3.8	电磁场方程 (Elmer FEM 端)	9
4	数值算法	10
4.1	PIMPLE 算法流程	10
4.2	界面捕捉方法	11
4.2.1	VOF 标准方法 (MULES)	11
4.2.2	isoAdvector 方法	11
4.2.3	子循环策略	12
4.3	OpenFOAM-Elmer 耦合机制	12
4.3.1	EOF 库架构	12
4.3.2	耦合变量	12
4.3.3	耦合策略	12
4.3.4	Elmer 端多区域修改	13
4.4	多区域共轭传热	13
4.5	时间步控制	14
5	关键代码结构分析	14
5.1	求解器入口 (以完整 MHD-CHT 求解器为例)	14
5.2	流体区求解流程 (solveMhdFluid.H)	15
5.3	MHD 耦合数据流	16
5.4	相分数更新触发机制	16
5.5	CPAD 后处理	17
6	求解器变体对比	17
6.1	命名与特性矩阵	17
6.2	源码复用关系	17
7	编译、配置与使用方法	18
7.1	环境前提	18
7.2	建议编译顺序	18
7.3	EOF 和 OpenFOAM v1912 特殊处理	19
7.4	随附 ESR 算例	19
7.5	关键字典	20
7.6	运行检查清单	20
8	MPI 通信修正	20
9	源码审计发现与验证边界	21
10	数值方法总结	21
10.1	空间离散	21
10.2	时间离散	22
10.3	线性求解器	22
11	结论	22

1 引言

1.1 项目背景

本求解器集合面向电渣重熔 (Electroslag Remelting, ESR) 过程中多物理场耦合问题的数值模拟。ESR 过程涉及电磁场、温度场、流场和相界面的强耦合，具体物理现象包括：

- 交流/直流电流通过渣池和金属熔池产生的焦耳热效应
- 洛伦兹力驱动的磁流体动力学流动
- 电极熔化、熔滴形成与滴落的多相流过程
- 渣-金属两相界面的演变
- 固体区域（结晶器壁、电极）的热传导
- 流体-固体界面的共轭传热

1.2 依赖项目

本求解器集合是在多个开源项目基础上组合开发而成：

- **OpenFOAM** (openfoam.com v1912)：基础 CFD 求解框架
- **Elmer FEM**：有限元多物理场求解器，用于求解电磁场方程
- **EOF-Library**：OpenFOAM-Elmer 耦合接口库 [1]
- **VoFLibrary**：DLR 开发的 VOF 方法扩展库
- **isoAdvector**：几何界面重构方法 [2]
- **varRhoTurbVOF**：变密度湍流 VOF 模型 [3]
- **cpad**：连续相区域检测后处理库

1.3 审计范围与阅读约定

本文采用以下证据优先级：

1. 求解器主程序及其实际 `#include` 调用链；
2. `Make/files`、`Make/options` 和算例字典；
3. 仓库 README 与上游论文；
4. 仅用于解释背景、但未在仓库中直接实现的连续方程。

需要特别区分两条求解器族：

- **单区域 varRho** 族是不可压、等温两相流，使用显式密度加权湍流黏度；MHD 版本只接收洛伦兹力 $\mathbf{J} \times \mathbf{B}$ ，不求解温度，也不接收焦耳热。

- **多区域 chtMultiRegion 族**是可压、非等温两相流与固体热传导耦合，使用以下可压两相运输模型：

```
compressibleInterPhaseTransportModel
```

该族名称中没有 varRho，不能把两种湍流闭合混为一谈。

本仓库包含不少从 OpenFOAM v1912 和第三方项目复制后再组合的文件。因此这里所谓“继承关系”主要是源码复制、头文件复用和链接依赖关系，不一定是 C++ 类继承关系。

1.4 命名约定

求解器命名遵循以下约定：

表 1: 求解器命名前缀含义

前缀/后缀	含义
mhd	通过 EOF 库与 Elmer FEM 耦合，包含 MHD 效应
varRho	采用变密度湍流模型 (Fan & Anglart, 2019)
interIso / interFlow	使用几何界面运输框架；具体方法由字典选择，算例选择 isoAdvector
Cpad	包含连续相区域检测用于后处理
_mod	修改版：电导率混合前限幅，并对低相分数单元施加相别截止
chtMultiRegion	多区域共轭传热（流体区 + 固体区）

表中 MHD 只表示存在 EOF 电磁耦合。单区域 mhdVarRho 实现只接收 JxB；多区域 MHD-CHT 实现同时接收 JxB 和 JH。此外，README 的 17 项中只列出 compressibleInterFlowCpad，但源码中也存在其无 Cpad 基础目录。

2 求解器架构总览

2.1 求解器分类

根据 README 正式列出的名称，17 个求解器可分为以下层级：

1. **可压、非等温两相基线**：compressibleInterFlowCpad（仓库还保留未列入 17 个正式名称的 compressibleInterFlow 基础目录）。

2. **不可压、等温、可变密度湍流**：

```
varRhoInterFoam
varRhoInterFoamCpad
varRhoInterIsoFoam
varRhoInterIsoFoamCpad
```

3. **单区域 MHD-varRho**：

```

mhdVarRhoInterFoam
mhdVarRhoInterFoamCpad
mhdVarRhoInterIsoFoam
mhdVarRhoInterIsoFoamCpad
mhdVarRhoInterIsoFoamCpad_mod

```

4. 多区域共轭传热: chtMultiRegionInterFoam、chtMultiRegionInterIsoFoam。

5. MHD-CHT 完整耦合:

```

mhdChtMultiRegionInterFoam
mhdChtMultiRegionInterFoamCpad
mhdChtMultiRegionInterIsoFoam
mhdChtMultiRegionInterIsoFoamCpad
mhdChtMultiRegionInterIsoFoamCpad_mod

```

2.2 目录结构

```

1 .
2 |-- solver/                                # solver sources
3 |   |-- chtMultiRegionFoam/
4 |   |   |-- fluid/                        # fluid equations
5 |   |   |-- solid/                       # solid equations
6 |   |   |-- include/                     # time-step helpers
7 |   |   |-- mhdChtMultiRegion*/
8 |   |   `-- chtMultiRegion*/
9 |   |-- compressibleInterFlow/
10 |   `-- mhdVarRho*/
11 |-- libs/
12 |   |-- eof/                             # OpenFOAM-Elmer coupling
13 |   |-- VoF/                             # v1912 VOF headers
14 |   |-- VoF_Library/                     # geometric VOF library
15 |   |-- cpad/                             # README only in this snapshot
16 |   |-- varRhoIncompressible/
17 |   `-- commSplit_modified_1912/
18 `-- examples/

```

Listing 1: 项目顶层目录结构

注记 2.1. 多个 *Cpad* 求解器的 *Make/options* 实际引用 *libs/of-cpad-library/src/lnInclude* 并链接 *-lcpad*, 但本仓库快照中只有 *libs/cpad/README.md*, 没有对应源码和 *Make* 文件。因此 *Cpad* 变体不能仅靠当前仓库独立编译; 必须另行取得 *of-cpad-library*, 并保持目录名与 *Make/options* 一致, 或修改这些路径。

3 控制方程

本节以最完整的求解器 *mhdChtMultiRegionInterIsoFoamCpad_mod* 为参照, 描述所有物理模型的控制方程。

3.1 相分数方程 (VOF 方法)

连续层面以 α_1 表示第一相体积分数。无相变、共享速度场时，基本输运方程可写为

$$\frac{\partial \alpha_1}{\partial t} + \nabla \cdot (\alpha_1 \mathbf{U}) = \alpha_1 \nabla \cdot \mathbf{U} + S_\alpha. \quad (1)$$

右端源项在代码中由 `dgdt` 构造为线性化的 `Sp/Su`。完整 CHT-Iso 路径实际调用

```
1 #include "alphaSuSp.H"
2 advector->advect(Sp, (Su + divU*min(alpha1(), scalar(1))))();
3 rhoPhi = advector->getRhoPhi(rho1, rho2);
4 alphaPhi10 = advector->alphaPhi();
```

因此几何界面通量由运行时选择的 `advectionSchemes` 对象生成，而不是把人工压缩项直接离散进这个头文件。算例在 `system/mixture/fvSolution` 中选择 `advectionScheme isoAdvect` 和 `reconstructionSchemes plicRDF`。

两相满足约束条件：

$$\alpha_1 + \alpha_2 = 1 \quad (2)$$

混合物的热物性采用体积加权：

$$\rho = \alpha_1 \rho_1 + \alpha_2 \rho_2 \quad (3)$$

3.2 MULES 人工压缩与几何输运的区别

`interFoam` 路径的 MULES 方程包含常见压缩通量

$$\nabla \cdot [\alpha_1(1 - \alpha_1)\mathbf{U}_c], \quad \mathbf{U}_c \parallel \mathbf{n}_\alpha, \quad (4)$$

代码通过 `cAlpha`、`icAlpha` 和 `scAlpha` 构造面压缩通量 `phic`，再由 MULES 保证有界性。而本文主线的 `InterIso/InterFlow` 路径使用几何重构与几何面通量。`fvSolution` 中仍可看到 `cAlpha` 等兼容项，但真正采用何种输运算算法由 `advectionScheme` 决定，不能只根据字段名判断。

3.3 质量守恒方程

多区域可压路径采用

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{U}) = 0 \quad (5)$$

在求解器实现中，连续方程通过相分数通量和压力方程共同满足。`contErr.H` 的实际定义还扣除了两个相上的 `fvOptions` 质量源：

$$\text{contErr} = \frac{\partial \rho}{\partial t} + \nabla \cdot \phi_\rho - S_{\rho,1} - S_{\rho,2}. \quad (6)$$

单区域 `varRho` 路径则假定两相各自不可压，使用 $\nabla \cdot \mathbf{U} = 0$ 和 $\rho = \alpha_1 \rho_1 + (1 - \alpha_1) \rho_2$ ；其“`varRho`”是湍流模型显式使用局部混合密度，并不是热力学可压缩。

3.4 动量方程（含 MHD 洛伦兹力）

忽略离散修正项后，主线求解器的动量方程可概括为

$$\frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \otimes \mathbf{U}) - \text{contErr } \mathbf{U} = -\nabla p_{\text{rgh}} - gh \nabla \rho + \nabla \cdot \boldsymbol{\tau} + \mathbf{f}_\sigma + \overline{\mathbf{J} \times \mathbf{B}} + \mathbf{S}_U. \quad (7)$$

其中各项含义：

- $p_{\text{rgh}} = p - \rho \mathbf{g} \cdot \mathbf{x}$: 减去静水压力的动压力
- $\boldsymbol{\tau} = \mu_{\text{eff}} [\nabla \mathbf{U} + (\nabla \mathbf{U})^T - \frac{2}{3}(\nabla \cdot \mathbf{U})\mathbf{I}]$: 有效黏性应力张量
- $\mathbf{f}_\sigma$: 表面张力（CSF 模型）
- $\mathbf{J} \times \mathbf{B}$: 洛伦兹力（MHD 耦合项，由 Elmer FEM 提供）
- $\mathbf{S}_U$: 其他动量源项（如 Darcy 阻尼、MRF 等）

在代码中，动量方程的离散形式为（参见 `mhdUEqn.H`）：

```
1 fvVectorMatrix UEqn
2 (
3     fvm::ddt(rho, U) + fvm::div(rhoPhi, U)
4     - fvm::Sp(contErr, U)
5     + MRF.DDt(rho, U)
6     + turbulence.divDevRhoReff(U)
7     ==
8     fvOptions(rho, U)
9     + JxB      // MHD Lorentz force source term
10 );
```

其中 JxB 的 OpenFOAM 维度为

$$[1 \ -2 \ -2 \ 0 \ 0 \ 0 \ 0],$$

即 N/m^3 。`setRegionMhdFluidFields.H` 直接执行 `JxB.field() = JxB_recv.field()`，当前实现没有再乘 α_1 ；因此 Elmer 侧映射范围和单位必须已经正确。

3.5 能量方程

3.5.1 流体区温度方程

该求解器没有调用标准 `EEqn.H`，而是使用自定义 `mhdTEqn.H`。令

$$\chi_{Cv} = \frac{\alpha_1}{C_{v,1}} + \frac{\alpha_2}{C_{v,2}}, \quad K = \frac{1}{2}|\mathbf{U}|^2,$$

则源码离散残差对应

$$\begin{aligned} \mathcal{R}_T = & \frac{\partial(\rho T)}{\partial t} + \nabla \cdot (\phi_\rho T) - \text{contErr } T - \nabla \cdot (\alpha_{\text{eff}} \nabla T) \\ & + \chi_{Cv} \left[p \nabla \cdot \mathbf{U} - \frac{p}{\rho} \text{contErr} + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\phi_\rho K) - \text{contErr } K + J_H \right] = S_T. \end{aligned} \quad (8)$$

其中：

- $K = \frac{1}{2}|\mathbf{U}|^2$: 动能
- α_{eff} : 有效热扩散系数（含湍流贡献）
- J_H : Elmer 返回的焦耳热字段，OpenFOAM 维度为 W/m^3
- $C_{v,1}$ 、 $C_{v,2}$: 两相的定容比热容
- S_T : 辐射等其他源项

关键源码为：

```

1 fvScalarMatrix TEqn
2 (
3     fvm::ddt(rho, T) + fvm::div(rhoPhi, T) - fvm::Sp(contErr, T)
4     - fvm::laplacian(turbulence.alphaEff(), T)
5     + (
6         (divU*p)() - contErr/rho*p
7         + (fvc::ddt(rho, K) + fvc::div(rhoPhi, K))() - contErr*K
8         + JH
9     )
10    *(
11        alpha1()/mixture.thermo1().Cv()()
12        + alpha2()/mixture.thermo2().Cv()()
13    )
14    ==
15    fvOptions(rho, T)
16 );

```

注记 3.1. 这里存在必须在生产计算前核对的符号问题： JH 以正号加入矩阵左侧括号，而固体方程用 $-JH$ 放在左侧。按通常 *fvMatrix* 记号，将流体式移到显式右端后，正的 JH 表现为温度汇；与此同时 *setRegionMhdFluidFields.H* 又把负的 JH 截为零。这与“焦耳热为正热源”的物理预期不一致。本文按源码原样记录，不把该项擅自改写成已验证的正热源。

3.5.2 固体区能量方程

固体区域求解焓形式热传导方程（参见 *solveMhdSolid.H*）：

$$\frac{\partial(\beta_v \rho h)}{\partial t} = \nabla \cdot (\beta_v \alpha_h \nabla h) + J_H + S_h. \quad (9)$$

其中 h 为热力学模型返回的比焓， β_v 为可选体积分数（缺省为 1）， $\alpha_h = \kappa/C_p$ ；各向异性材料使用张量形式。源码中的 $-JH$ 位于左侧，因此移项后是正热源。

3.6 压力方程（PIMPLE 算法）

压力方程通过动量预测通量与两相可压缩性修正构造。采用 $p_{\text{rgh}} = p - \rho gh$ 作为求解变量，源码算子形式为

$$\sum_{i=1}^2 \frac{\max(\alpha_i, 0)}{\rho_i} C_i(p_{\text{rgh}}) + \nabla \cdot \phi_{H/A} - \nabla \cdot (r_{AU,f} \nabla p_{\text{rgh}}) = 0. \quad (10)$$

其中 $r_{AU} = 1/A_U$, $\phi_{H/A}$ 包含动量预测通量、时间修正、MRF、重力和表面张力, C_i 包含第 i 相的 $\partial\rho_i/\partial t$ 、 $\psi_i\partial p_{rgh}/\partial t$ 及对流修正。非正交循环结束后更新

$$\phi = \phi_{H/A} + \phi_p, \quad \mathbf{U} = \mathbf{H}/A_U + r_{AU} \text{reconstruct}(\dots),$$

随后按热力学压缩率修正 ρ_i , 并执行 $p = \max(p_{rgh} + \rho gh, p_{\min})$ 。

3.7 湍流模型

3.7.1 CHT 路径的可压缩两相湍流

完整 CHT 求解器构造 `compressibleInterPhaseTransportModel`, 并由 `constant/mixture/turbulenceProperties` 选择 RAS、LES 或 laminar。示例选择 `RNGkEpsilon`。其有效动力黏度可概括为

$$\mu_{\text{eff}} = \mu + \mu_t \quad (11)$$

3.7.2 单区域 varRho 路径

根据 Fan & Anglart (2019)[3] 的工作, 变密度湍流模型显式引用密度场以考虑等温变密度效应。关键修改:

$$\mu = \rho\nu, \quad \mu_t = \rho\nu_t, \quad \mu_{\text{eff}} = \rho(\nu + \nu_t) \quad (12)$$

这使得湍流黏度直接与局部混合密度耦合, 适用于不可压、等温但混合密度随相分数变化的两相流。该实现不在 `mhdChtMultiRegionInterIsoFoamCpad_mod` 的调用链中。

在代码中 (`varRhoIncompressibleTurbulenceModel.C`):

```
1 Foam::tmp<Foam::volScalarField>
2 Foam::varRhoIncompressibleTurbulenceModel::mu() const
3 {
4     return rho_*nu(); // Dynamic viscosity = density * kinematic viscosity
5 }
6
7 Foam::tmp<Foam::volScalarField>
8 Foam::varRhoIncompressibleTurbulenceModel::mut() const
9 {
10    return rho_*nut(); // Turbulent viscosity = density * turbulent nu
11 }
```

3.8 电磁场方程 (Elmer FEM 端)

仓库本身不离散 Maxwell 方程; 它通过 EOF 把电导率发送给 Elmer。随附算例的 `case_eof_harm.sif` 选择 `WhitneyAVHarmonicSolver`, 频率为 50 Hz。用相量 $\hat{\mathbf{A}}$ 、 $\hat{\mathbf{V}}$ 表示时, 常用涡流 A - V 形式可写成

$$\nabla \times (\mu^{-1} \nabla \times \hat{\mathbf{A}}) + i\omega\sigma\hat{\mathbf{A}} + \sigma\nabla\hat{\mathbf{V}} = \hat{\mathbf{J}}_{\text{ext}}, \quad (13)$$

$$\nabla \cdot [\sigma (-i\omega\hat{\mathbf{A}} - \nabla\hat{\mathbf{V}})] = 0. \quad (14)$$

具体规范、边界条件和复数符号约定由 Elmer 实现决定。由 $\hat{\mathbf{B}} = \nabla \times \hat{\mathbf{A}}$ 、 $\hat{\mathbf{J}} = \sigma \hat{\mathbf{E}}$ 可计算周期平均量

$$\overline{\mathbf{f}_L} = \frac{1}{2} \text{Re} \left(\hat{\mathbf{J}} \times \hat{\mathbf{B}}^* \right), \quad (15)$$

$$\overline{q_J} = \frac{1}{2} \text{Re} \left(\hat{\mathbf{J}} \cdot \hat{\mathbf{E}}^* \right). \quad (16)$$

电导率 σ 在 VOF 框架下为两相混合值：

$$\sigma = \alpha_1 \sigma_1 + (1 - \alpha_1) \sigma_2 \quad (17)$$

CHT 的 `_mod` 版本先限幅，再执行额外截止：

```
1 // 原始版本
2 elcond = alpha1 * elCondListByPhase[0] + (1 - alpha1) * elCondListByPhase[1];
3
4 // _mod 版本：增加相分数截断
5 alpha1f = min(max(alpha1, scalar(0)), scalar(1));
6 elcond = alpha1f * elcond1 + (1 - alpha1f) * elcond2;
7 if (alpha1[celli] < 0.1)
8 {
9     elcond[celli] = elcond2.value();
10 }
```

这等价于

$$\sigma(\alpha_1) = \begin{cases} \sigma_2, & \alpha_1 < 0.1, \\ \bar{\alpha}_1 \sigma_1 + (1 - \bar{\alpha}_1) \sigma_2, & \alpha_1 \geq 0.1, \end{cases} \quad \bar{\alpha}_1 = \min(1, \max(0, \alpha_1)). \quad (18)$$

单区域 `mhdVarRhoInterIsoFoamCpad_mod` 使用的截止值则是 0.5，两个 `_mod` 实现并不相同。

4 数值算法

4.1 PIMPLE 算法流程

求解器采用 PIMPLE (PISO-SIMPLE 混合) 算法处理压力-速度耦合。对于多区域求解器，采用外层 PIMPLE 循环和内层 PISO 循环的嵌套结构。

图 1: 单时间步内 PIMPLE 算法流程图 (完整 MHD-CHT 求解器)

1. 启动阶段创建所有区域和场, 向 `Elmer` 发送初始电导率, 并阻塞接收初始 `JxB_recv/JH_recv`。
2. 每个时间步先读取控制字典, 计算流体 Courant 数、界面 Courant 数和固体扩散数, 再调整 Δt 。
3. 进入外层 PIMPLE 循环。第一次外迭代调用 `advectord->surf().reconstruct()`。
4. 对每个流体区调用 `solveMhdFluid.H`: 相分数输运 $\rightarrow$ 相通量修正 $\rightarrow$ 动量方程 $\rightarrow$ 温度方程 $\rightarrow$ 压力校正 $\rightarrow$ 湍流修正。
5. 对每个固体区调用 `solveMhdSolid.H`, 求解含 JH 的焓方程。
6. 第一次外迭代后可进入 `energyCoupling` 循环; 流体区把 `frozenFlow` 设为真, 只重复温度方程, 固体区重复焓方程。
7. **PIMPLE 全部结束以后**才比较 $\max |\alpha_1 - \alpha_{1,old}|$ 。达到阈值时发送新电导率, 接收下一次使用的 `JxB/JH`。
8. 写出 OpenFOAM 时间步; Cpad 版本随后生成连续相报告。

注记 4.1. 因此流体和固体方程使用的是“最近一次成功 *Elmer* 更新”的电磁场, 不是当前 *PIMPLE* 外迭代中即时重算的场。该分区顺序属于显式/滞后耦合; *energyCoupling* 循环也不会触发新的电磁求解。

4.2 界面捕捉方法

4.2.1 VOF 标准方法 (MULES)

标准 VOF 求解器 (`interFoam` 族) 使用 MULES (Multidimensional Universal Limiter for Explicit Solution) 方法保证相分数的有界性和守恒性:

- 显式时间推进
- 通量限制器保证 $\alpha \in [0, 1]$
- 支持子循环以放宽 Courant 数限制

4.2.2 isoAdvectord 方法

`InterIso/InterFlow` 路径通过 VoF-Library 的运行时选择机制装配几何输运; 本算例明确选择 `isoAdvectord`[2]。其基本步骤为:

1. **界面重构**: 在每个包含界面的网格单元内, 通过 α 值和梯度信息重构界面等值面
2. **通量计算**: 基于重构的等值面, 计算通过每个面的精确几何通量
3. **时间积分**: 沿时间方向精确积分通量

在代码中 (`alphaEqn.H`):

```

1 // isoAdvectord 相输运
2 #include "alphaSuSp.H"
3 advectord->advect(Sp, (Su + divU*min(alpha1(), scalar(1))))();
4
5 // 更新密度通量
6 rhoPhi = advectord->getRhoPhi(rho1, rho2);
7 alphaPhi10 = advectord->alphaPhi();

```

4.2.3 子循环策略

为满足界面 Courant 数限制 (通常 $Co_\alpha < 0.5$)，相分数方程可在每个时间步内进行 N 个子循环：

$$\Delta t_{\text{sub}} = \frac{\Delta t}{N_{\text{sub}}} \quad (19)$$

每次子循环求解一次相方程，最终通量为各子循环通量之和：

$$\phi_{\rho, \text{total}} = \sum_{k=1}^{N_{\text{sub}}} \frac{\Delta t_{\text{sub}}}{\Delta t} \phi_{\rho}^{(k)} \quad (20)$$

4.3 OpenFOAM-Elmer 耦合机制

4.3.1 EOF 库架构

EOF (Elmer-OpenFOAM coupler for electromagnetics and fluid dynamics) 库 [1] 实现了基于 MPI 的 OpenFOAM 与 Elmer FEM 的耦合。耦合架构如下：

- **通信方式**：OpenFOAM 与 Elmer 以 MPMD 方式在同一个 `mpirun` 中启动，EOF 在共享 MPI 作业中建立通信关系
- **网格映射**：OpenFOAM 有限体积网格 $\leftrightarrow$ Elmer 有限元网格之间的数据插值
- **重叠检测**：通过包围盒 (bounding box) 比较确定各 MPI 分区间的网格重叠关系
- **数据插值**：在重叠区域进行单元中心数据的插值传输

4.3.2 耦合变量

表 2: OpenFOAM-Elmer 耦合变量交换

方向	物理量	变量名
OF $\rightarrow$ Elmer	电导率	elcond
Elmer $\rightarrow$ OF	洛伦兹力 (矢量)	JxB_recv $\rightarrow$ JxB
Elmer $\rightarrow$ OF	焦耳热 (标量)	JH_recv $\rightarrow$ JH

4.3.3 耦合策略

所有 MHD 主程序都采用相分数变化触发策略。变量名 `maxRelDiff` 容易误导：代码计算的是最大绝对差，不是相对差：

```

1 scalar maxRelDiff_local = (max(mag(alpha1_old - alpha1))).value();
2
3 if (maxRelDiff_local > maxRelDiff && (maxRelDiff < SMALL || maxRelDiff+SMALL <= 1.0)
4 )
5 {
6     updateElmerSolution = true;
7 }
8
9 if (updateElmerSolution || !runTime.run())
10 {
11     alpha1_old = alpha1;
12     // 发送电导率, 接收洛伦兹力和焦耳热
13     sending.sendScalar(elcond);
14     receiving.recvVector(JxB_recv);
15     receiving.recvScalar(JH_recv);
16 }

```

启动时总会交换一次。时间循环内, 只有绝对变化超过阈值或程序准备结束时才继续 Elmer。注释给出的语义是: 1 近似只算一次, $0 < \varepsilon < 1$ 按阈值更新, 0 尽可能频繁更新; 严格来说, 当变化恰为零时阈值 0 也不会触发。

4.3.4 Elmer 端多区域修改

为支持多区域耦合, 对 EOF 库的 Elmer 端进行了以下修改 (elmer_mr_solver/):

- 增加了 Bodies 关键字, 支持将不同物理体的场变量分别发送到不同的 OpenFOAM 区域
- 修改了 Elmer2OpenFOAM.F90 和 OpenFOAM2Elmer.F90 中数组索引, 以支持多体-多区域映射

4.4 多区域共轭传热

多区域求解器管理多个流体区域 (fluidRegions) 和固体区域 (solidRegions), 通过以下机制实现共轭传热:

- 流体-固体界面温度场和热通量通过 OpenFOAM 的可压缩 “coupled baffle mixed temperature” 边界条件耦合
- 独立的时间步控制 (流体 Courant 数 + 固体扩散数)
- 区域间能量耦合迭代 (energyCoupling 循环)

理想连续条件为

$$T_f = T_s, \quad -\mathbf{n}_f \cdot k_f \nabla T_f = \mathbf{n}_s \cdot k_s \nabla T_s. \quad (21)$$

算例在 changeDictionaryDict 中把 mixture_to_*, skin_to_* 和 mould_to_* 设置为以下类型:

```
compressible::turbulentTemperatureCoupledBaffleMixed
```

由边界条件迭代满足温度/热流耦合。

能量耦合循环的代码逻辑 (mhdChtMultiRegionInterIsoFoamCpad_mod.C):

```

1 // 能量耦合子循环
2 if (!oCorr && nOuterCorr > 1)
3 {
4     loopControl looping(runTime, pimple, "energyCoupling");
5     while (looping.loop())
6     {
7         forAll(fluidRegions, i)
8         {
9             frozenFlow = true; // 冻结流动, 仅求解能量
10            #include "solveMhdFluid.H"
11        }
12        forAll(solidRegions, i)
13        {
14            #include "solveMhdSolid.H"
15        }
16    }
17 }

```

4.5 时间步控制

- 流体 Courant 数 Co : 由所有流体区取最大值;
- 界面 Courant 数 Co_α : 只在界面附近统计并打印;
- 固体扩散数 Di : 由所有固体区取最大值。

`setMultiRegionDeltaT.H` 的实际缩放因子为

$$r_f = \min \left(\min \left(\frac{Co_{\max}}{Co}, 1 + 0.1 \frac{Co_{\max}}{Co} \right), 1.2 \right), \quad (22)$$

$$\Delta t_{\text{new}} = \min \left[\min \left(r_f, \frac{Di_{\max}}{Di} \right) \Delta t, \Delta t_{\max} \right]. \quad (23)$$

注记 4.2. 虽然主程序读取并打印 $maxAlphaCo$ 和 Co_α , 当前 `setMultiRegionDeltaT.H` 并没有使用它们。因此示例中的 $maxAlphaCo$ 0.9 不会直接限制全局时间步; 界面稳定性主要依赖固定的 `nAlphaSubCycles` 和用户自行选择的 $maxCo/maxDeltaT$ 。这是文档和算例参数容易产生误解的地方。

5 关键代码结构分析

5.1 求解器入口 (以完整 MHD-CHT 求解器为例)

```

1 int main(int argc, char *argv[])
2 {
3     #include "createTime.H"
4     #include "createMeshes.H"
5     #include "createFields.H"
6     #include "createMhdFields.H" // includes initial EOF send/receive
7
8     while (runTime.run())

```

```

9      {
10         #include "readPIMPLEControls.H"
11         #include "compressibleMultiRegionCourantNo.H"
12         #include "alphaMultiRegionCourantNo.H"
13         #include "solidRegionDiffusionNo.H"
14         #include "setMultiRegionDeltaT.H"
15
16         ++runTime;
17
18         for (int oCorr=0; oCorr<nOuterCorr; ++oCorr)
19         {
20             if (firstIter)
21                 advector->surf().reconstruct();
22
23             forAll(fluidRegions, i)
24                 #include "solveMhdFluid.H"
25
26             forAll(solidRegions, i)
27                 #include "solveMhdSolid.H"
28
29             if (!oCorr && nOuterCorr > 1) { ... }
30         }
31
32         // Update rho/p, then conditionally exchange fields with Elmer
33         forAll(fluidRegions, i) { ... send/receive ... }
34         forAll(solidRegions, i) { ... receive JH ... }
35
36         runTime.write();
37         // Cpad report follows runTime.write()
38     }
39 }

```

Listing 2: mhdChtMultiRegionInterIsoFoamCpad_mod.C 主程序结构

5.2 流体区求解流程 (solveMhdFluid.H)

```

1  if (finalIter)
2      mesh.data::add("finalIteration", true);
3
4  if (frozenFlow)
5  {
6      // 冻结流动模式：仅求解温度方程（用于能量耦合迭代）
7      #include "mhdTEqn.H"
8  }
9  else
10 {
11     // 正常求解模式
12     // Step 1: 求解相分数方程（含子循环）
13     #include "compressibleAlphaEqnSubCycle.H"
14
15     // Step 2: 湍流相通量修正
16     turbulence.correctPhasePhi();

```

```

17
18 // Step 3: 求解动量方程 (含 JxB 洛伦兹力源项)
19 #include "mhdUEqn.H"
20
21 // Step 4: 求解温度方程 (含 JH 焦耳热源项)
22 #include "mhdTEqn.H"
23
24 // Step 5: PISO 压力校正循环
25 for (int corr=0; corr<nCorr; corr++)
26     #include "pEqn.H"
27
28 // Step 6: 湍流模型修正
29 turbulence.correct();
30 }

```

Listing 3: solveMhdFluid.H 流体区求解流程

5.3 MHD 耦合数据流

图 2 展示了 MHD 耦合的完整数据流。

OpenFOAM (有限体积)	Elmer (有限元)
由 $\alpha_1, \sigma_1, \sigma_2$ 构造 elcond	→ 接收并映射电导率 $\sigma(\mathbf{x})$
保持 CFD 场不变并等待	求解谐波电磁问题, 计算 $\overline{\mathbf{J} \times \mathbf{B}}$ 与 $\overline{q_J}$
接收 JxB_recv 和 JH_recv	← 按 Bodies/目标变量映射并发送
下一时间步把 JxB 加入动量方程, JH 加入流体/固体能量方程	等待下一次 sendStatus/elcond

图 2: MHD 耦合数据流与时间滞后

5.4 相分数更新触发机制

MHD 耦合求解器中的一个关键优化是仅当相分数变化足够大时才触发 Elmer 求解:

```

1 // sendUpdateRegionMhdFluidFields.H
2 scalar maxRelDiff_local = (max(mag(alpha1_old - alpha1))).value();
3
4 if (maxRelDiff_local > maxRelDiff
5     && (maxRelDiff < SMALL || maxRelDiff + SMALL <= 1.0))
6 {
7     updateElmerSolution = true;
8 }
9
10 if (updateElmerSolution || !runTime.run())
11 {
12     alpha1_old = alpha1;
13     // 仅此时触发 Elmer 求解和数据交换
14     sending.sendStatus(runTime.run());
15     sending.sendScalar(elcond);
16     receiving.sendStatus(runTime.run());

```

```

17     receiving.recvVector(JxB_recv);
18     receiving.recvScalar(JH_recv);
19 }

```

5.5 CPAD 后处理

CPAD (Continuous Phase Area Detection) 库在每次时间步结束时调用，用于识别和标记连续相区域，便于后处理分析（如识别孤立液滴、统计相分布等）：

```

1 // CPAD 后处理 (mhdVarRhoInterIsoFoamCpad_mod.C)
2 #include "getLocalToGlobalFaceArray.h"
3 FoamCpad::cpad cpas(mixture, runTime);
4 cpas.appendTimeStepReport(runTime.value(), procFaceToGlobalFaceList);

```

6 求解器变体对比

6.1 命名与特性矩阵

表 3 汇总了 README 列出的 17 个正式变体。源码中还保留以下未列入清单的无 Cpad 基础目录，表中一并列出：

```
compressibleInterFlow
```

表 3: 求解器特性矩阵

求解器名称	MHD	CHT	isoAdvectord	varRho	Cpad	_mod
compressibleInterFlow			✓			
compressibleInterFlowCpad			✓		✓	
varRhoInterFoam				✓		
varRhoInterFoamCpad				✓	✓	
varRhoInterIsoFoam			✓	✓		
varRhoInterIsoFoamCpad			✓	✓	✓	
mhdVarRhoInterFoam	✓			✓		
mhdVarRhoInterFoamCpad	✓			✓	✓	
mhdVarRhoInterIsoFoam	✓		✓	✓		
mhdVarRhoInterIsoFoamCpad	✓		✓	✓	✓	
mhdVarRhoInterIsoFoamCpad_mod	✓		✓	✓	✓	✓
chtMultiRegionInterFoam		✓				
chtMultiRegionInterIsoFoam		✓	✓			
mhdChtMultiRegionInterFoam	✓	✓				
mhdChtMultiRegionInterFoamCpad	✓	✓			✓	
mhdChtMultiRegionInterIsoFoam	✓	✓	✓			
mhdChtMultiRegionInterIsoFoamCpad	✓	✓	✓		✓	
mhdChtMultiRegionInterIsoFoamCpad_mod	✓	✓	✓		✓	✓

6.2 源码复用关系

族	基础目录/模型	增量组合
可压两相	compressibleInterFlow	VoF-Library 几何输运；Cpad 版本增加后处理。

族	基础目录/模型	增量组合
不可压 varRho	varRhoInterFoam	varRhoInterIsoFoam 增加几何输运; Cpad 版本增加后处理。
单区域 MHD	mhdVarRhoInterFoam	在 varRho 族上链接 EOF 并加入 JxB; Iso/Cpad/_mod 继续叠加。
多区域 CHT	chtMultiRegionFoam 与 chtMultiRegionInterFoam	chtMultiRegionInterIsoFoam 替换界面输运路径。
MHD-CHT	MHD-CHT 基线目录	链接 EOF, 流体接收 JxB/JH, 固体接收 JH; Iso/Cpad/_mod 继续叠加。

7 编译、配置与使用方法

7.1 环境前提

- 目标版本是 **OpenFOAM.com v1912**。更高版本的 API、库名、边界条件和 `wmake` 路径很可能需要移植。
- Elmer FEM 必须启用 MPI; OpenFOAM 与 Elmer 必须链接到同一 MPI 实现, 否则 MPMD 作业通常会在启动或通信阶段失败。
- `etc/.bashrc` 只是模板。需要把 `MYPATH` 设置为仓库根目录, 把 `FOAM_PATH` 和 `ELMER_PATH` 指向实际安装。

当前模板还包含两个路径/命令问题, 使用前应修正:

- `EOF_MR_SOLVER_SRC` 应指向 `$MYPATH/libs/eof/elmer_mr_solver`;
- `eofCompile()` 中 `wmake $EOF_OF_SRC/coupleElmer` 会重复目录名, 应直接对 `$EOF_OF_SRC` 执行 `wmake`。

7.2 建议编译顺序

下面命令表达依赖顺序; 具体 `wmake` 语法可按本机 OpenFOAM 环境调整:

```

1 export REPO=/path/to/OpenFOAM--MHD--VoF-Solver-Collection-master
2 source /opt/openfoam/OpenFOAM-v1912/etc/bashrc
3 export PATH=/opt/elmerfem/elmer-mpi-install/bin:$PATH
4
5 # 1. Geometric VOF library, required by InterIso/InterFlow solvers
6 wmake libso "$REPO/libs/VoF_Library/VoF"
7
8 # 2. varRho turbulence library, only for varRho solver family
9 wmake libso "$REPO/libs/varRhoIncompressible"
10
11 # 3. EOF OpenFOAM-side coupling library
12 wmake libso "$REPO/libs/eof/coupleElmer"
13
14 # 4. EOF Elmer-side multi-region solvers

```

```

15 elmerf90 -o "$REPO/libs/eof/elmer_mr_solver/Elmer2OpenFOAM.so" \
16 -J"$REPO/libs/eof/elmer_mr_solver" \
17 "$REPO/libs/eof/elmer_mr_solver/Elmer2OpenFOAM.F90"
18 elmerf90 -o "$REPO/libs/eof/elmer_mr_solver/OpenFOAM2Elmer.so" \
19 -J"$REPO/libs/eof/elmer_mr_solver" \
20 "$REPO/libs/eof/elmer_mr_solver/OpenFOAM2Elmer.F90"
21
22 # 5. Target solver
23 wmake "$REPO/solver/chtMultiRegionFoam/"\
24 "mhdChtMultiRegionInterIsoFoamCpad_mod"

```

使用 Cpad 变体前还要安装 of-cpad-library, 生成 libcpad, 并使其头文件路径与各 Make/options 中的 libs/of-cpad-library/src/lnInclude 一致。

7.3 EOF 和 OpenFOAM v1912 特殊处理

1. 备份 \$FOAM_SRC/Pstream/mpi。
2. 检查 libs/commSplit_modified_1912/install_mod.sh 中的路径后, 再复制补丁并重编译 Pstream。该脚本会修改 OpenFOAM 安装树, 不应在未备份时直接运行。
3. 将 libs/eof/elmer_mr_solver/SOLVER.KEYWORDS_mod 中新增的 Bodies 和 Body i Use Target Variable 关键字合并到 Elmer 安装的 SOLVER.KEYWORDS。
4. 确保 Elmer2OpenFOAM.so 和 OpenFOAM2Elmer.so 能被 Elmer 的 Procedure 路径找到。

7.4 随附 ESR 算例

算例位于 examples/mhdChtMultiRegionInterIsoFoam_mod, 包含一个流体区 mixture 和两个固体区 skin/mould。推荐流程如下:

1. 从 meshElmer/mesh.grd 生成 Elmer 网格, 确认存在 mesh.header、mesh.nodes、mesh.elements 和 mesh.boundary。
2. 检查 system/controlDict 的求解器名称、时间步、maxCo、maxDi 和输出频率。
3. 令根目录及每个区域的 decomposeParDict 使用相同 numberOfSubdomains=N, 并把 Allrun.eof 中 cpu_cores 同步为 N。
4. 检查 case_eof_harm.sif 中频率、电流密度、Elmer body 编号、Bodies 映射和目标变量顺序。
5. 执行预处理, 再以 MPMD 方式同时启动 OpenFOAM 与 Elmer。

```

1 cd "$REPO/examples/mhdChtMultiRegionInterIsoFoam_mod"
2 ./Allrun.pre
3
4 # Allrun.eof performs Elmer partitioning and decomposePar -allRegions,
5 # then launches the equivalent of:
6 mpirun -n N mhdChtMultiRegionInterIsoFoamCpad_mod -parallel \
7 : -n N ElmerSolver_mpi case_eof_harm.sif

```

7.5 关键字典

文件	作用
constant/regionProperties	声明 <code>mixture</code> 为流体区, <code>skin/mould</code> 为固体区。
constant/mixture/ thermophysicalProperties	相名、表面张力、最低压力和 <code>maxRelDiff</code> 。
constant/mixture/thermo.* 或相热物 性字典	两相 ρ 、 C_v 、黏度、热导率和电导率 <code>elcond</code> 。
system/mixture/fvSolution	界面输运、重构方法、PIMPLE 次数、线性求解器和松弛。
system/mixture/fvSchemes	时间、梯度、对流、扩散和电导率插值格式。
system/cpad	目标相、阈值以及按 <code>faces/edges/points</code> 判断连通。
case_eof_harm.sif	Elmer 谐波电磁方程、材料、边界、电导率输入及 <code>JxB/JH</code> 输出映射。

7.6 运行检查清单

- 启动日志应先出现每个流体区发送 `elcond`, 随后接收 `JxB_recv/JH_recv`, 固体区接收 `JH_recv`。
- 检查 `alpha.metal` 的最小/最大值、总质量、压力下限、Courant 数和固体扩散数。
- 检查 `JxB` 的单位为 N/m^3 、`JH` 的单位为 W/m^3 , 并验证流体温升方向。
- 若长时间看不到“FOAM: Continue Elmer!”, 检查 `maxRelDiff` 是否过大; 若每步都重算, 则检查阈值和相分数噪声。
- 结束时确认 OpenFOAM 与 Elmer 都正常退出。任一侧提前退出都可能让另一侧阻塞在发送/接收调用。

8 MPI 通信修正

EOF 库需要修改 OpenFOAM v1912 的 MPI 通信模块 (`commSplit_modified_1912/`), 因为 OpenFOAM 的 `Pstream` 实现与 EOF 库的 MPI 通信假设存在兼容性问题。修改涉及以下文件:

- `PstreamGlobals.C / PstreamGlobals.H`: 全局 MPI 通信器管理
- `UPstream.C`: 并行数据流基类
- `UIPread.C`: 并行输入读取
- `UOPwrite.C`: 并行输出写入
- `allReduce.H / allReduceTemplates.C`: 全局归约操作

安装脚本 `install_mod.sh` 自动将这些文件复制到 OpenFOAM 安装目录的对应位置。

9 源码审计发现与验证边界

发现	影响	建议
流体 mhdTEqn.H 中 +JH 位于左侧	按通常矩阵符号会表现为热汇，与固体方程及物理预期不一致	用零流动、均匀正 JH 的单元算例检查温升方向后再生产使用。
maxAlphaCo 未进入多区域时间步控制头文件	界面 Courant 数只打印，不限制全局时间步	保守设置 maxCo/maxDeltaT，或修改时间步控制。
电磁更新在时间步末发生	当前时间步使用上一次 Elmer 场，属于滞后分区耦合	对快速界面运动降低 maxRelDiff，并做时间步敏感性研究。
maxRelDiff 实际是最大绝对相分数差	名称可能让用户误认为是相对误差	把阈值解释为 $[0, 1]$ 相分数的绝对变化。
_mod 版使用硬截止	$\sigma(\alpha)$ 在阈值处不连续，可能使电磁解跳变	对 0.1 (CHT) 或 0.5 (单区域) 做参数敏感性分析。
Cpad 源码缺失且 Make 路径指向另一目录名	Cpad 求解器在当前快照中无法直接链接	补齐上游库，统一路径后再编译。
etc/.bashrc 含路径和函数拼接错误	照抄模板会找不到 EOF 多区域源码或重复目录	按本章给出的真实仓库路径手动配置。
仓库作者声明未做数学验证/确认	结果不能仅凭程序收敛视为可信	至少完成守恒、网格、时间步、耦合阈值和实验基准验证。

10 数值方法总结

10.1 空间离散

随附算例而非求解器硬编码的主要选择为：

- 梯度：cellLimited leastSquares 1.0；
- 动量对流：Gauss linearUpwind grad(U)；
- 温度、动能、压力和密度相关对流：Gauss linear；
- 湍流标量：Gauss upwind；
- Laplacian 与法向梯度：linear corrected/corrected；
- 界面：plicRDF 重构和 isoAdvector 输运；
- 表面张力由 mixture 模型提供，动量预测中通过 mixture.surfaceTensionForce() 加入。

10.2 时间离散

- 算例所有区域使用 Euler 一阶隐式时间离散；
- 相分数设置 `nAlphaSubCycles 2`；
- MULES 兼容头文件支持 Crank–Nicolson，但与多重相分数子循环不兼容；主线几何运输的可用组合还需以 VoF-Library 实现为准。

10.3 线性求解器

- 流体压力 `p_rgh`: GAMG + Gauss–Seidel smoother；
- 速度: `smoothSolver` + Gauss–Seidel；
- T 与湍流标量: `smoothSolver` + symGauss–Seidel；
- 固体焓 h : PCG + DIC；
- 相分数由几何运输/限制器更新，不走普通标量线性求解器。

11 结论

本文档从源码和算例还原了求解器的真实控制流。该集合的主要工程组合包括：

1. **多物理场组合**：VOF 多相流、MHD 电磁力/焦耳热和多区域共轭传热；`varRho` 是另一条单区域求解器族
2. **自适应耦合策略**：基于相分数变化的自适应 MHD 耦合触发机制，有效降低了计算开销
3. **尖锐界面捕捉**：通过 `isoAdvector` 几何重构方法实现优于标准 VOF 的界面分辨率
4. **多区域扩展**：对 EOF 库进行了多区域/多体扩展，支持流体-固体多区域 MHD 耦合
5. **模块化复用**：通过主程序、头文件和链接库组合形成多个复杂度级别的求解器变体

该仓库提供了有价值的 ESR 电磁–热–流–界面耦合原型，但当前快照仍存在第三方库缺失、环境脚本错误、显式滞后耦合和焦耳热符号疑点。在补齐依赖并完成守恒性、网格、时间步、耦合阈值和实验验证之前，应把它视为研究代码而不是已认证的工业预测工具。

参考文献

- [1] J. Vencels, P. Råback, V. Geža, “EOF-Library: Open-source Elmer FEM and OpenFOAM coupler for electromagnetics and fluid dynamics,” *SoftwareX*, vol. 9, pp. 68–72, 2019. <https://doi.org/10.1016/j.softx.2019.01.007>
- [2] J. Roenby, H. Bredmose, H. Jasak, “A computational method for sharp interface advection,” *Royal Society Open Science*, vol. 3, no. 11, p. 160405, 2016. <https://doi.org/10.1098/rsos.160405>

- [3] W. Fan, H. Anglart, “varRhoTurbVOF: A new set of volume of fluid solvers for turbulent isothermal multiphase flows in OpenFOAM,” *Computer Physics Communications*, p. 106876, 2019. <https://doi.org/10.1016/j.cpc.2019.106876>
- [4] K. J. Vachaparambil, K. E. Einarsrud, “Comparison of Surface Tension Models for the Volume of Fluid Method,” *Processes*, vol. 7, no. 8, p. 542, 2019. <https://doi.org/10.3390/pr7080542>